SISTEMAS DISTRIBUIDOS · CURSO 2026/2027 · UNIVERSIDAD DE ALICANTE

Guía de pruebas
Water Management Network

Cómo ejecutar y comprobar todo lo implementado hasta ahora: sockets, estados, riego simulado y órdenes de la central

Laura Caturla Martín · Borja Lara Zarco · 13 de octubre de 2026

Preparación

REQUISITOS

- Python 3 (probado con 3.12). No hace falta instalar nada: solo se usa la biblioteca estándar (`socket`, `threading`, `sqlite3`, `tkinter`, `queue`, `random`).
- 7 terminales (PowerShell o la terminal de VS Code) abiertas en la carpeta raíz del repositorio, `SD`. La terminal 1 se mueve a `Central`.
- Para empezar con la BD vacía, como en las capturas, renombrar o borrar `Central/database.db` antes de la prueba P1. **Se pierden las estaciones guardadas.** Si no se borra, las estaciones registradas aparecen al principio en gris.

ESCENARIO Y PUERTOS

Terminal	Proceso	Escucha en	Se conecta a
1	WM_Central	9999	(BD SQLite local)
2 / 3 / 4	WM_WS_M de WS_01 / 02 / 03	8000 / 8001 / 8002 (para su Engine)	Central en 127.0.0.1:9999
5 / 6 / 7	WM_WS_E de WS_01 / 02 / 03	—	Su Monitor en 8000 / 8001 / 8002 (el broker 9092 aún no se usa)

Para probar en dos ordenadores, sustituir `127.0.0.1` por la IP de la máquina donde corre la central (o el Monitor), y permitir esos puertos en el cortafuegos.

COMANDOS DE CADA MÓDULO

Módulo	Uso
WM_Central	`python WM_Central.py <puerto_escucha>`
WM_WS_M	`python WM_WS_M.py <puerto_engine> <ip_central> <puerto_central> <id_ws> <ubicacion>`
WM_WS_E	`python WM_WS_E.py <ip_broker> <puerto_broker> <ip_monitor> <puerto_monitor>`

Teclado	Efecto
Engine	`R <s>` regar · `P` parar riego · `K` provocar fuga · `O` resolver fuga · `FIN` salir
Monitor	`FIN` desconectarse de la central
Central	Panel: pinchar una fila o usar el desplegable, y los botones **Iniciar riego**, **Bloquear** y **Activar**

> Las capturas y salidas de consola de esta guía son de una ejecución real de las pruebas, en este orden y en el mismo escenario. Los clics en los botones del panel se automatizaron con un script; el efecto es el mismo que pulsarlos a mano. Las salidas de consola se han recortado a la parte relevante de cada prueba.

P1 Arranque de la central

> **Objetivo.** Comprobar que WM_Central arranca, crea las tablas de la BD y muestra el panel.

PASOS

Terminal 1

```
cd Central
python WM_Central.py 9999
```

RESULTADO ESPERADO

- Se abre la ventana con la tabla de estaciones, la barra de órdenes y el registro de eventos.
- Con la BD vacía, la tabla aparece vacía y el registro indica `0 estaciones cargadas`.
- Si la BD ya tenía estaciones, aparecen todas en gris (*Desconectada*).

RESULTADO OBTENIDO



Figura 1. Central recién arrancada con la base de datos vacía.

P2 Registro de estaciones (Monitor + Engine)

> **Objetivo.** Comprobar que cada Monitor se autentica en la central y que su Engine se conecta al Monitor y recibe el ID.

PASOS

Terminal 2

```
python WM_WS_M.py 8000 127.0.0.1 9999 WS_01 "Bosque de los Dioses"
```

Terminal 3

```
python WM_WS_M.py 8001 127.0.0.1 9999 WS_02 "Patio de Armas"
```

Terminal 4

```
python WM_WS_M.py 8002 127.0.0.1 9999 WS_03 "Invernadero"
```

Terminal 5

```
python WM_WS_E.py 127.0.0.1 9092 127.0.0.1 8000
```

Terminal 6

```
python WM_WS_E.py 127.0.0.1 9092 127.0.0.1 8001
```

Terminal 7

```
python WM_WS_E.py 127.0.0.1 9092 127.0.0.1 8002
```

RESULTADO ESPERADO

- Cada Monitor recibe `ACK#REGISTER_OK#WS_0X`.
- Cada Engine muestra `ID asignado a este engine: WS_0X`.
- Las tres estaciones aparecen en verde (*Disponible*) y el registro muestra los tres `[REGISTRO OK]`.

RESULTADO OBTENIDO



Figura 2. Tres estaciones registradas y disponibles.

Terminal 2 · WM_WS_M (WS_01)

```
****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****
Conexión establecida con ('127.0.0.1', 9999)
[RESPUESTA CENTRAL]: ACK#REGISTER_OK#WS_01
Introduce comando/estado o 'FIN' para salir: [ENGINE] Esperando conexión del engine en el puerto 8000

[ENGINE] Engine conectado desde ('127.0.0.1', 56331)
```

Terminal 5 · WM_WS_E (WS_01)

```
****** WM_WS_E - ENGINE DE ESTACIÓN DE RIEGO ****
Comandos:
  R <segundos> = iniciar riego manual   P = parar riego
  K = provocar avería (KO)   O = resolver avería (OK)   FIN = salir

[MONITOR] Conectado al monitor en ('127.0.0.1', 8000)
[MONITOR] ID asignado a este engine: WS_01
```

P3 Autenticación: ID ya conectado

> **Objetivo.** Comprobar que la central rechaza un segundo Monitor con un ID que ya está conectado.

PASOS

Terminal 8 (nueva)

```
python WM_WS_M.py 8010 127.0.0.1 9999 WS_01 Impostor
```

RESULTADO ESPERADO

- El nuevo Monitor recibe `NACK#REGISTER_KO` y termina solo.
- La WS_01 original sigue en verde: el intento no le afecta.

RESULTADO OBTENIDO

Terminal 8 · Monitor rechazado

```
****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****
Conexión establecida con ('127.0.0.1', 9999)
Registro rechazado o conexión cerrada por la central: NACK#REGISTER_KO#WS_01#ID ya conectado
```

Terminal 1 · WM_Central

```
[REGISTRO KO] 'WS_01' ya está conectada, se rechaza ('127.0.0.1', 64611)
```

P4 Fuga simulada y resolución

> **Objetivo.** Comprobar que el Monitor detecta el KO del Engine, avisa a la central y avisa también cuando se resuelve.

PASOS

1. En el Engine de WS_01 (terminal 5) escribir `K` y pulsar Enter.
2. Esperar 2 segundos y comprobar el panel.
3. Escribir `O` y pulsar Enter.

RESULTADO ESPERADO

- Tras `K`, WS_01 pasa a rojo (*Fuga*) en un segundo como máximo.
- Tras `O`, vuelve a verde.
- El Monitor solo envía mensajes en los cambios de estado, no en cada `KO`.

RESULTADO OBTENIDO

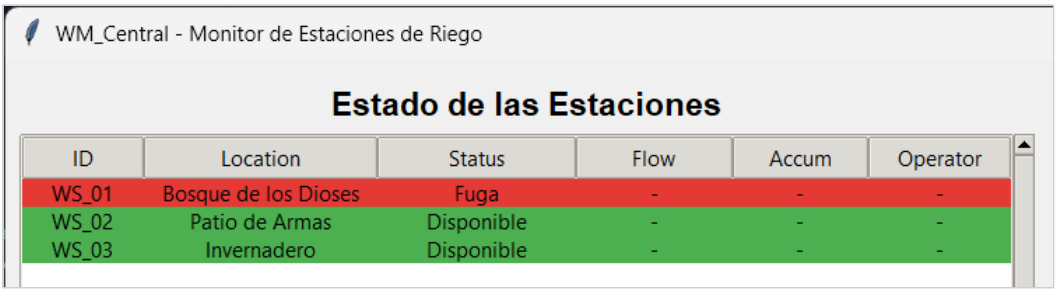


Figura 3. WS_01 en fuga tras pulsar K en su Engine.

Terminal 2 · WM_WS_M (WS_01)

```
****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****
Conexión establecida con ('127.0.0.1', 9999)
[RESPUESTA CENTRAL]: ACK#REGISTER_OK#WS_01
Introduce comando/estado o 'FIN' para salir: [ENGINE] Esperando conexión del engine en el puerto 8000

[ENGINE] Engine conectado desde ('127.0.0.1', 64602)

[ENGINE] Respuesta de avería recibida: KO

[ESTADO] WS_01 pasa a LEAK
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_01#LEAK

[ENGINE] Respuesta de avería recibida: KO

[ENGINE] Respuesta de avería recibida: KO

[ESTADO] WS_01 pasa a AVAILABLE
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_01#AVAILABLE
```

P5 Caída y reconexión del Engine

> **Objetivo.** Comprobar que, si el Engine deja de responder, el Monitor lo trata como avería y que se recupera cuando el Engine vuelve.

PASOS

1. En el Engine de WS_03 (terminal 7) escribir `FIN`. También vale cerrar la terminal o pulsar Ctrl+C.
2. Comprobar el panel.
3. Volver a lanzar el Engine:

Terminal 7

```
python WM_WS_E.py 127.0.0.1 9092 127.0.0.1 8002
```

RESULTADO ESPERADO

- Al caer el Engine, WS_03 pasa a rojo y el Monitor queda esperando una reconexión.
- Al relanzar el Engine, recibe de nuevo el ID y WS_03 vuelve a verde.

RESULTADO OBTENIDO

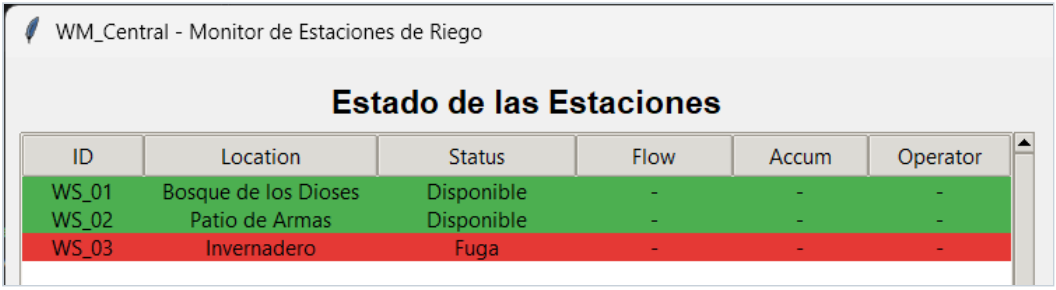


Figura 4. WS_03 en rojo tras apagar su Engine.

Terminal 4 · WM_WS_M (WS_03)

```
****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****
Conexión establecida con ('127.0.0.1', 9999)
[RESPUESTA CENTRAL]: ACK#REGISTER_OK#WS_03
Introduce comando/estado o 'FIN' para salir: [ENGINE] Esperando conexión del engine en el puerto 8002

[ENGINE] Engine conectado desde ('127.0.0.1', 64606)

[ENGINE] El engine no responde o se ha desconectado

[ESTADO] WS_03 pasa a LEAK
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_03#LEAK
[ENGINE] Esperando reconexión del engine en el puerto 8002

[ENGINE] Engine conectado desde ('127.0.0.1', 64612)

[ESTADO] WS_03 pasa a AVAILABLE
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_03#AVAILABLE
```

P6 Riego manual desde el Engine

> **Objetivo.** Comprobar la simulación de riego (caudal, volumen y fin del riego) y las condiciones que lo impiden.

PASOS

En el Engine de WS_02 (terminal 6), escribir estos comandos uno tras otro:

Comando	Qué se prueba
`R 4`	Riego de 4 s que termina solo (`TIEMPO`).
`R 30`, luego `R 10`	No se puede lanzar un segundo riego mientras hay uno en curso.
`P`	Parada manual (`MANUAL`).
`R 30`, luego `K`	Una fuga corta el riego en el acto (`FUGA`).
`R 5`	Con fuga no se puede regar.
`O`	Se resuelve la fuga.

RESULTADO ESPERADO

- Durante el riego, cada segundo aparece una línea con el caudal (15-25 l/min) y el volumen acumulado.
- Al terminar se muestra el motivo y un resumen con el volumen total y la duración.
- Con `FIN` durante un riego, este se cierra con motivo `APAGADO` antes de salir.

> El riego todavía no se refleja en el panel de la central: el Engine lo comunicará por Kafka. Por eso en esta prueba WS_02 sigue en verde mientras riega, y solo cambia a rojo un momento por el `K`.

RESULTADO OBTENIDO

Terminal 6 · WM_WS_E (WS_02)

```
[RIEGO] Electroválvula ABIERTA. Operario: LOCAL | Límite: 4 s
[RIEGO] Caudal: 17.02 l/min | Acumulado: 0.28 l | 1/4 s
[RIEGO] Caudal: 17.95 l/min | Acumulado: 0.58 l | 2/4 s
[RIEGO] Caudal: 15.65 l/min | Acumulado: 0.84 l | 3/4 s
[RIEGO] Caudal: 21.81 l/min | Acumulado: 1.21 l | 4/4 s
[RIEGO] Electroválvula CERRADA. Motivo: TIEMPO
[RESUMEN] Operario: LOCAL | Volumen total: 1.21 l | Duración: 4 s
[RIEGO] Electroválvula ABIERTA. Operario: LOCAL | Límite: 30 s
[RIEGO] Caudal: 16.23 l/min | Acumulado: 0.27 l | 1/30 s
[RIEGO] Caudal: 21.31 l/min | Acumulado: 0.63 l | 2/30 s
[RIEGO] No se puede regar: ya hay un riego en curso
[RIEGO] Electroválvula CERRADA. Motivo: MANUAL
[RESUMEN] Operario: LOCAL | Volumen total: 0.63 l | Duración: 2 s
[RIEGO] Electroválvula ABIERTA. Operario: LOCAL | Límite: 30 s
[RIEGO] Caudal: 23.65 l/min | Acumulado: 0.39 l | 1/30 s
[RIEGO] Caudal: 22.72 l/min | Acumulado: 0.77 l | 2/30 s
[AVERÍA] Fuga simulada: se responderá KO al monitor
[AVERÍA] Riego en curso cortado por la fuga
[RIEGO] Electroválvula CERRADA. Motivo: FUGA
[RESUMEN] Operario: LOCAL | Volumen total: 0.77 l | Duración: 2 s
[RIEGO] No se puede regar: hay una avería (fuga) detectada
[AVERÍA] Fuga resuelta: se responderá OK al monitor
```

P7 Órdenes de la central: iniciar riego y bloquear

> **Objetivo.** Comprobar la autorización de un riego desde la central y el bloqueo de una estación (punto 11 del enunciado).

PASOS

1. En el panel, pinchar la fila de WS_01: el desplegable *Estación* cambia a WS_01.
2. Con *Duración* 30, pulsar **Iniciar riego**.
3. Elegir WS_02 en el desplegable y pulsar **Bloquear**.

RESULTADO ESPERADO

- El registro muestra `[RIEGO] Autorizado en 'WS_01'` y la orden `REGAR#WS_01#CENTRAL#30` marcada como pendiente de Kafka.
- WS_02 pasa a naranja (*Fuera de Servicio*) y se registra la orden `BLOQUEAR#WS_02`.

RESULTADO OBTENIDO

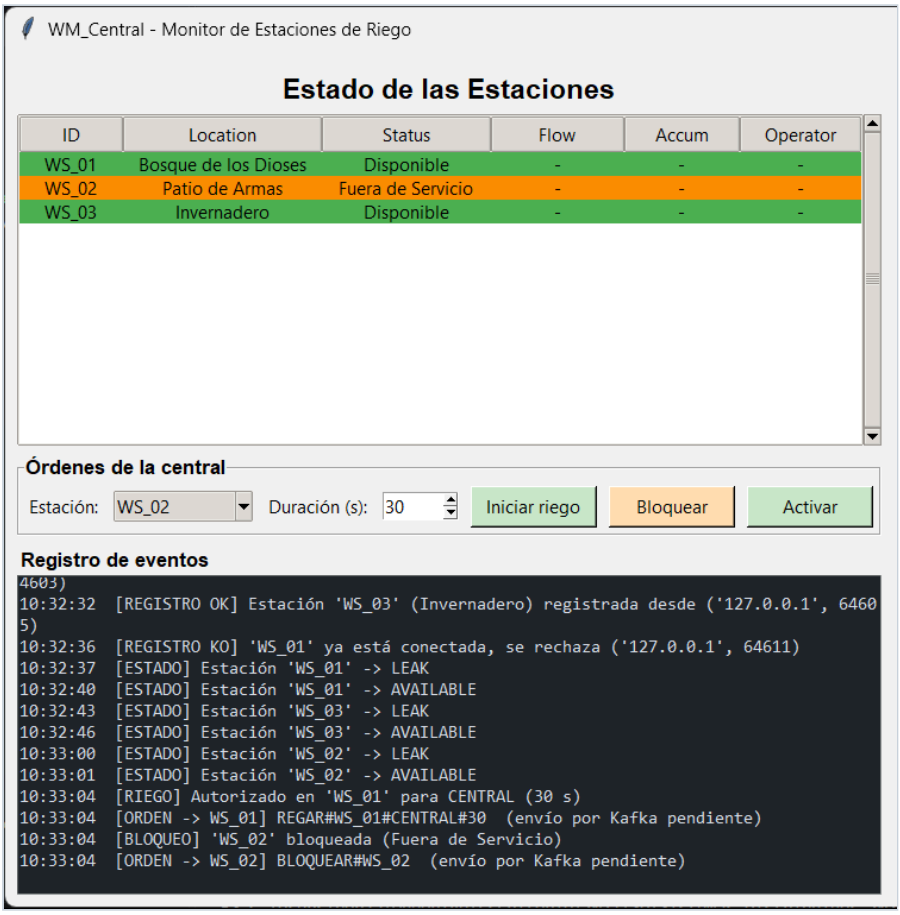


Figura 5. WS_02 bloqueada y riego autorizado en WS_01.

P8 Comprobaciones antes de autorizar un riego

> **Objetivo.** Comprobar que la central deniega el riego en las estaciones que no están disponibles, usando la orden sobre TODAS.

PASOS

1. En el Engine de WS_01 (terminal 5) escribir `K` para provocar una fuga.
2. En el panel, elegir **TODAS** en el desplegable y pulsar **Iniciar riego**.
3. Escribir `O` en el Engine de WS_01.

RESULTADO ESPERADO

- WS_01: *Denegado, la estación tiene una fuga*.
- WS_02: *Denegado, la estación está fuera de servicio*.
- WS_03: *Autorizado* y orden `REGAR`.
- La orden sobre TODAS solo afecta a las estaciones conectadas.

RESULTADO OBTENIDO

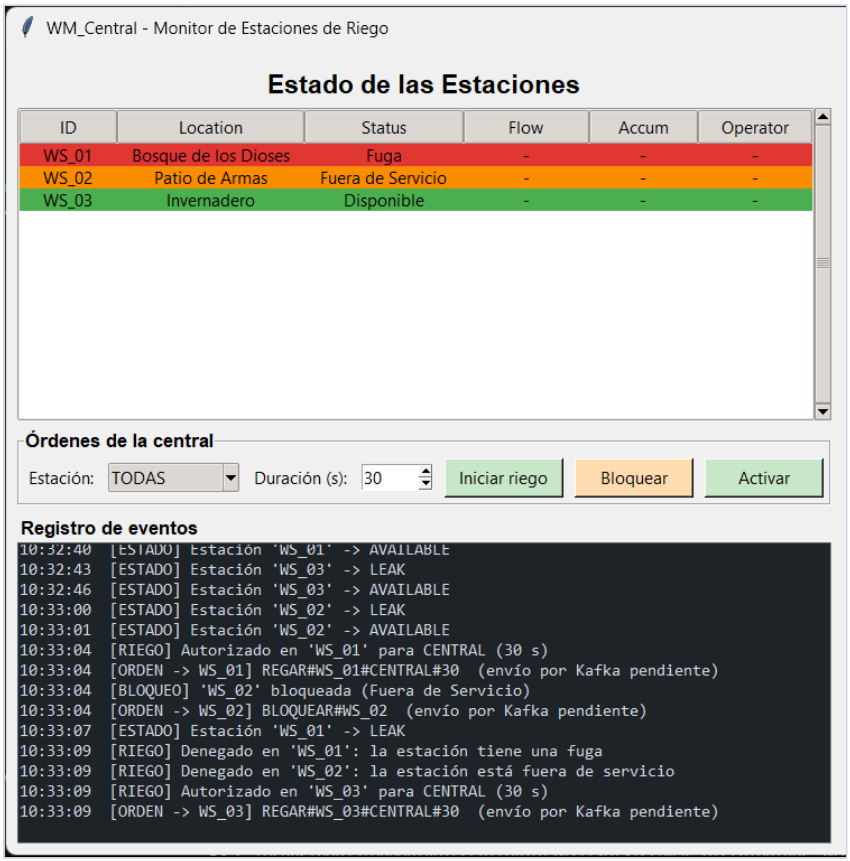


Figura 6. Riego sobre TODAS: dos denegaciones y una autorización.

Terminal 1 · WM_Central

```
[RIEGO] Denegado en 'WS_01': la estación tiene una fuga
[RIEGO] Denegado en 'WS_02': la estación está fuera de servicio
[RIEGO] Autorizado en 'WS_03' para CENTRAL (30 s)
[ORDEN -> WS_03] REGAR#WS_03#CENTRAL#30  (envío por Kafka pendiente)
```

P9 Fuga en una estación bloqueada

> **Objetivo.** Comprobar que una fuga se muestra aunque la estación esté bloqueada y que, al resolverse, vuelve a Fuera de Servicio y no a Disponible.

PASOS

1. En el Engine de WS_02 (bloqueada) escribir `K`.
2. Después escribir `O`.

RESULTADO ESPERADO

- Con `K`, WS_02 pasa de naranja a rojo: la avería tiene prioridad sobre el bloqueo.
- Con `O`, vuelve a naranja, porque sigue bloqueada por la central.

RESULTADO OBTENIDO

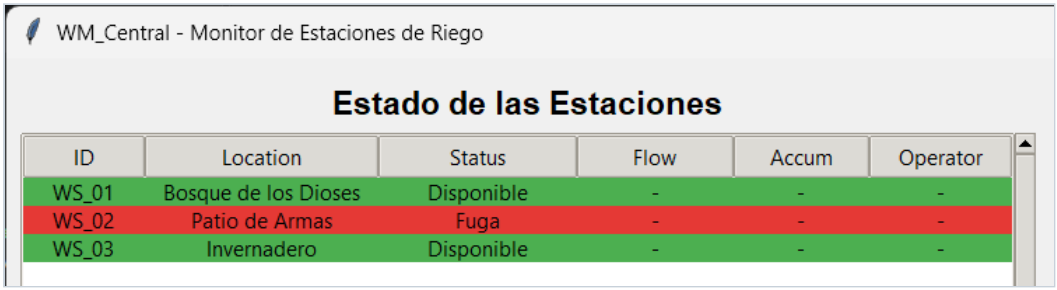


Figura 7. Fuga en WS_02 bloqueada.

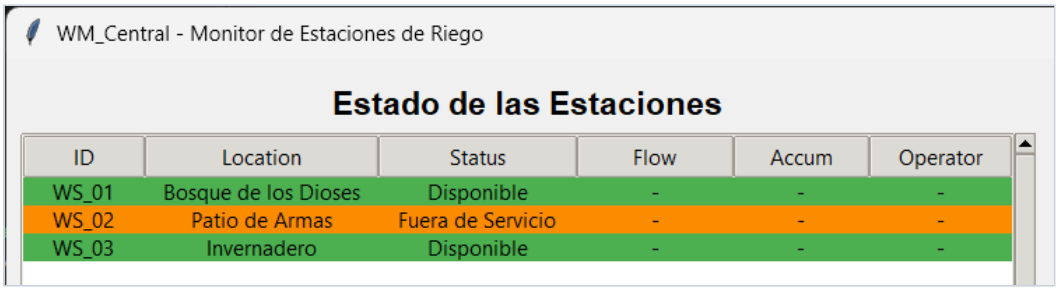


Figura 8. Fuga resuelta: WS_02 vuelve a naranja.

P10 Activar estaciones

> **Objetivo.** Comprobar que la orden Activar devuelve las estaciones bloqueadas a Disponible.

PASOS

1. Elegir **TODAS** y pulsar **Activar**.

RESULTADO ESPERADO

- WS_02 vuelve a verde. Las demás siguen como estaban.
- Se registra una orden `ACTIVAR` por cada estación conectada.

RESULTADO OBTENIDO



Figura 9. Todas las estaciones disponibles tras Activar.

P11 Desconexión de un Monitor

> **Objetivo.** Comprobar que la central marca una estación como desconectada cuando pierde su Monitor.

PASOS

1. En el Monitor de WS_02 (terminal 3) escribir `FIN`.
2. Repetir la prueba cerrando la terminal o con Ctrl+C. El resultado debe ser el mismo.

RESULTADO ESPERADO

- WS_02 pasa a gris (*Desconectada*).
- Su Engine se queda reintentando la conexión cada 3 s.

RESULTADO OBTENIDO



Figura 10. WS_02 desconectada.

Terminal 1 · WM_Central

```
[FIN] La estación 'WS_02' ha enviado FIN
[DESCONEXION] Estación 'WS_02' marcada como OFFLINE
```

P12 Reinicio de la central

> **Objetivo.** Comprobar que la central conserva las estaciones en la BD y las muestra como desconectadas hasta que vuelvan a registrarse (punto 1 de la mecánica).

PASOS

1. Cerrar todos los Engines y Monitores con `FIN`.
2. Cerrar la ventana de la central y volver a lanzarla:

Terminal 1 (carpeta Central)

```
python WM_Central.py 9999
```

RESULTADO ESPERADO

- Las tres estaciones aparecen en gris.
- Si se vuelve a lanzar un Monitor, su estación pasa a verde.

RESULTADO OBTENIDO



Figura 11. Central reiniciada: estaciones recuperadas de la BD.

Terminal 1 · WM_Central

```
[BD] 3 estaciones cargadas de la base de datos (todas en OFFLINE)
('WS_01', 'Bosque de los Dioses', 'OFFLINE', None, None, None)
('WS_02', 'Patio de Armas', 'OFFLINE', None, None, None)
('WS_03', 'Invernadero', 'OFFLINE', None, None, None)
[STARTING] Iniciando WM_Central en 0.0.0.0:9999...
[LISTENING] Servidor a la escucha en 0.0.0.0
```

P13 Contenido de la base de datos

> **Objetivo.** Comprobar las tres tablas: estaciones, operarios y riegos.

PASOS

Con la central cerrada, desde la carpeta `Central`:

Terminal 1

```
python -c "import sqlite3; c=sqlite3.connect('database.db'); [print(t, c.execute('SELECT * FROM '+t).fetchall()) for t in ('estaciones','operarios','riegos')]"
```

RESULTADO ESPERADO

- `estaciones`: las estaciones registradas, con su ubicación.
- `operarios`: CENTRAL, FO_01, FO_02 y FO_03, que se dan de alta solos al arrancar.
- `riegos`: vacía. Se rellenará cuando el Engine confirme los riegos por Kafka.

RESULTADO OBTENIDO

Terminal 1

```
estaciones [('WS_01', 'Bosque de los Dioses', 'OFFLINE', None, None, None), ('WS_02', 'Patio de Armas', 'OFFLINE', None, None, None), ('WS_03', 'Invernadero', 'OFFLINE', None, None, None)]
operarios [('CENTRAL', 'Central de control'), ('FO_01', 'Operario 1'), ('FO_02', 'Operario 2'), ('FO_03', 'Operario 3')]
riegos []
```

Lo que todavía no se puede probar

Estas partes dependen de Kafka. El código ya está preparado (`publicar_evento` en el Engine, `enviar_orden` en la central, y `riego_iniciado`, `telemetria_recibida` y `riego_finalizado` en la central), pero hoy no se pueden ver de punta a punta:

- Que una orden **Iniciar riego** de la central haga regar de verdad al Engine. Hoy solo queda registrada como pendiente.
- Ver en el panel una estación *Regando* (verde parpadeando) con caudal, volumen acumulado y operario en tiempo real.
- El resumen final del riego que la central envía al operario, y los registros de la tabla `riegos`.
- Que **Bloquear** corte un riego en curso en el Engine.
- La aplicación del operario, WM_FO.

Lista de comprobación para la defensa

Prueba	Resultado esperado	OK
P1	La central arranca y muestra el panel	☐
P2	3 estaciones registradas en verde; cada Engine recibe su ID	☐
P3	ID duplicado rechazado con NACK	☐
P4	K → rojo, O → verde	☐
P5	Engine caído → rojo; relanzado → verde	☐
P6	Riego manual: TIEMPO, MANUAL, FUGA y rechazos	☐
P7	Riego autorizado y bloqueo en naranja	☐
P8	Riego sobre TODAS: denegado con fuga y bloqueada	☐
P9	Fuga en bloqueada: rojo y vuelta a naranja	☐
P10	Activar: vuelve a verde	☐
P11	FIN o Ctrl+C en el Monitor → gris	☐
P12	Central reiniciada: estaciones en gris	☐
P13	BD con estaciones, operarios y riegos	☐